Level 2
Hackathon – Fundamentals of Web Development
Project: Online Quiz Application

Student Details

Student Name	GOPIKA V
Register Number	34625U50012
Department	B.Sc Artificial Intelligence
ID	9A1335685CD82DA8B1745266DF56CIE6
College	MMES Women's Arts and Science College
Date of Submission	27/10/2026

Git-Hub Link: https://github.com/<your-username>/online-quiz-application

1. Problem Statement

Students often prepare for exams by solving practice questions from books or printed question papers. These sources do not tell the student whether an answer is right until someone checks it, they do not keep time, and they do not show the student which topics are weak. Teachers who conduct small class tests face the same trouble: preparing papers, collecting answer sheets and correcting each one by hand takes hours.

Why the application is needed: A web-based quiz lets a student practise at any time from a laptop or a mobile phone without installing anything. The browser marks each answer immediately, runs a timer to build exam speed, and finishes with a summary and a review of every question. This saves paper and time, avoids marking errors and keeps the student motivated by remembering the best score for each subject.

2. Objectives

- To design and develop a responsive Online Quiz Application using only HTML, CSS and JavaScript.
- To let the user enter a name and choose one of three categories: Artificial Intelligence, Web Development or Python Basics.
- To present six randomly ordered multiple-choice questions per attempt, with the options also shuffled.
- To give a 15-second countdown for every question and to give instant green/red feedback after each answer.
- To provide a one-time 50-50 lifeline that removes two wrong options.
- To display the final score, percentage, correct / wrong / timed-out counts and a question-by-question answer review.
- To store the best score of each category in the browser (localStorage) so that progress is remembered.

Expected outcome: A fully working, attractive quiz website that can be opened by double-clicking one file, that behaves correctly on desktop and phone screens, and that demonstrates the fundamentals of HTML structure, CSS styling and JavaScript logic.

3. Architecture

The Online Quiz Application is a client-side single-page application. Everything runs inside the browser; there is no server and no database. The page contains three screens (Welcome, Quiz and Result) and JavaScript displays only one of them at a time by adding or removing a hidden class. The project is organised into three files:

File	Responsibility
index.html	Skeleton of the Welcome, Quiz and Result screens, the header with the best-score badge, and the footer.
style.css	Dark theme, card layout, category buttons, option states, circular timer, score ring and the mobile layout.
script.js	Question bank, shuffling, timer, answer checking, 50-50 lifeline, score, review list and localStorage.

Technologies used and their roles

Technology	Role in the application
HTML5	Defines the structure: sections for each screen, input box, buttons, progress track, lists and semantic header, main and footer tags.
CSS3	CSS variables for the colour theme, Flexbox and Grid for layout, conic-gradient for the score ring, transitions and a media query for phones.
JavaScript (ES6)	Stores questions as objects, shuffles them, handles click events, runs the timer with setInterval, checks answers and updates the DOM.
Web Storage (localStorage)	Saves the best score of every category inside the user's browser.
VS Code, Chrome, GitHub	Code editor, testing browser and version control / hosting of the project.

User navigation flow

The user opens index.html and sees the Welcome screen. After typing a valid name and choosing a category, pressing Start Quiz opens the Quiz screen. Each question is answered (or times out), after which Next is pressed. When the last question is finished, the Result screen shows the score and the answer review. From there the user can Play Again with the same category or go back to change the category.

Application Flow Diagram

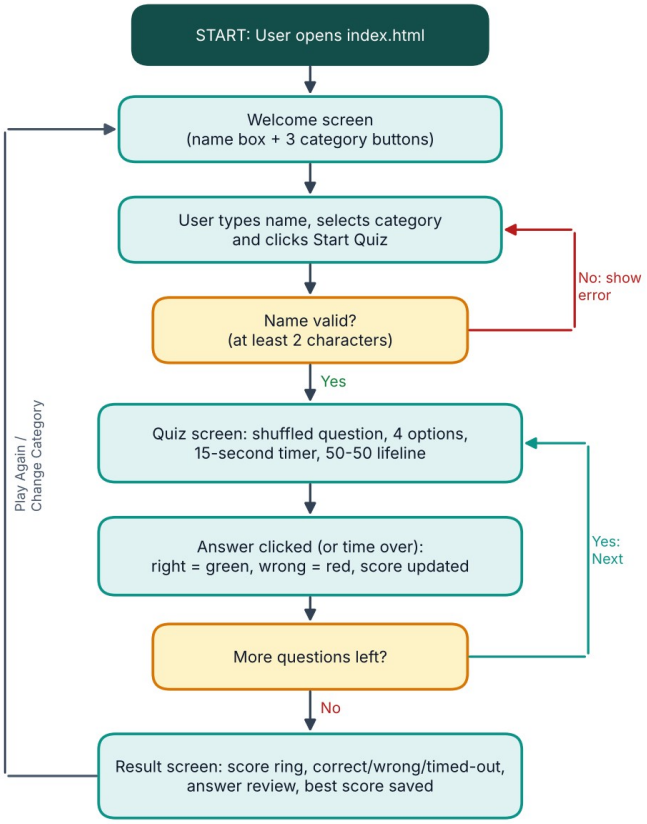


Figure 1: Application flow from the initial page to the final output

4. Practical Implementation

The application was developed step by step:

- Planning: the three screens, the three categories and the extra features (timer, lifeline, review, best score) were decided first.
- HTML: the three screens were written as separate section elements inside one main card.
- CSS: a dark theme with a teal accent was created using CSS variables, followed by option, timer and score-ring styles and a mobile media query.
- Data: 18 questions (6 per category) were stored as objects with the question, four options and the index of the correct option.
- Logic: functions were written for starting the quiz, loading a question, timing, answering, using the lifeline and showing the result.
- Testing: every path was tested in Chrome – empty name, correct answer, wrong answer, time-out, 50-50, restart and a 390 px mobile screen – and the browser console showed no errors.
- Publishing: the three files were uploaded to a GitHub repository.

4.1 Features Implemented

Feature	Purpose
Name validation	Starts the quiz only when a name of at least 2 characters is typed; otherwise an error message is shown.
Category selection	Lets the learner practise Artificial Intelligence, Web Development or Python Basics.
Shuffled questions and options	Order changes in every attempt, so answers cannot be memorised by position.
15-second timer	Circular timer that turns red in the last 5 seconds; on time-out the correct answer is revealed.
Instant feedback	Correct option turns green and a wrong choice turns red as soon as an option is clicked.
50-50 lifeline	Removes two wrong options once per attempt to help with a difficult question.
Live score and progress bar	Shows question number, current score and a gradient progress bar.
Result dashboard	Score ring with percentage, counts of correct / wrong / timed-out answers and a personalised message.
Answer review	Lists every question with the user's answer and the correct answer for revision.
Best score memory	Saves the highest score of each category in localStorage and shows it in the header.
Responsive design	Layout switches to a single column and smaller spacing on phone screens.

4.2 Source Code

The important parts of the three source files are shown below, each with an explanation of its functionality.

HTML

HTML builds the skeleton of the application. A header holds the logo and best-score badge. Inside the main element there are three section tags – welcomeScreen (name input, category buttons, error message and Start button), quizScreen (counter, live score, timer, progress track, question, option container, 50-50 and Next buttons) and resultScreen (score ring, statistics, answer review list and the two end buttons). The class hidden keeps inactive screens invisible. The stylesheet is linked in the head and the script is loaded at the end of the body so that all elements exist before the JavaScript runs.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Quiz Application</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="topbar">
    <div class="logo">Online Quiz Application</div>
    <div class="best" id="bestScore">Best: --</div>
```

```
</header>

<main class="card">

  <!-- Screen 1: Welcome -->
  <section id="welcomeScreen" class="screen">
    <h1>Test Your Tech Knowledge</h1>
    <p class="sub">Pick a category, answer 6 questions and beat your best score.</p>

    <label for="username">Your name</label>
    <input type="text" id="username" placeholder="e.g. Gopika" maxlength="20">

    <label>Choose a category</label>
    <div class="categories" id="categories">
      <button class="cat active" data-cat="ai">Artificial Intelligence</button>
      <button class="cat" data-cat="web">Web Development</button>
      <button class="cat" data-cat="python">Python Basics</button>
    </div>

    <p id="msg" class="msg"></p>
    <button id="startBtn" class="primary">Start Quiz</button>
  </section>

  <!-- Screen 2: Quiz -->
  <section id="quizScreen" class="screen hidden">
    <div class="meta">
      <span id="qCount">Q 1/6</span>
      <span id="scoreLive">Score: 0</span>
      <span class="timer" id="timer">15</span>
    </div>
    <div class="track"><div id="fill"></div></div>

    <h2 id="questionText"></h2>
    <div id="optionList" class="options"></div>

    <div class="actions">
      <button id="fiftyBtn" class="ghost">50-50</button>
      <button id="nextBtn" class="primary hidden">Next</button>
    </div>
  </section>

  <!-- Screen 3: Result -->
  <section id="resultScreen" class="screen hidden">
    <h1 id="resultTitle">Result</h1>
    <div class="ring"><span id="resultScore"></span></div>
    <p id="resultInfo" class="sub"></p>

    <div class="stats">
      <div><b id="statCorrect">0</b><small>Correct</small></div>
      <div><b id="statWrong">0</b><small>Wrong</small></div>
      <div><b id="statSkipped">0</b><small>Timed out</small></div>
    </div>

    <h3>Answer Review</h3>
    <ul id="reviewList" class="review"></ul>

    <div class="actions">
      <button id="retryBtn" class="ghost">Change Category</button>
      <button id="againBtn" class="primary">Play Again</button>
    </div>
  </section>

</main>
```

```
  <footer>Made by Gopika V | B.Sc Artificial Intelligence | MMES Women's Arts and Science College</footer>
  <script src="script.js"></script>
</body>
</html>
```

CSS

CSS gives the application its look. Colours are defined once as variables in :root, so the whole theme can be changed from one place. The body uses a radial gradient, the card has rounded corners and a shadow, the category buttons use a three-column grid, and the option buttons change border and background colour with the right and wrong classes. The score ring is drawn with a conic-gradient whose percentage is passed from JavaScript through the custom property --pct. A media query makes the layout friendly for small screens. Only the important rules are shown.

```
:root {
  --bg: #0f172a;
  --panel: #1e293b;
  --panel-2: #273449;
  --text: #e2e8f0;
  --muted: #94a3b8;
  --accent: #14b8a6;
  --accent-dark: #0d9488;
  --good: #22c55e;
  --bad: #ef4444;
  --warn: #f59e0b;
}

.card {
  width: 100%;
  max-width: 640px;
  background: var(--panel);
  border-radius: 18px;
  padding: 28px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, .45);
  border: 1px solid #334155;
}

.cat {
  background: var(--panel-2);
  color: var(--text);
  border: 2px solid transparent;
  border-radius: 12px;
  padding: 14px 8px;
  font-size: .85rem;
  cursor: pointer;
  transition: .2s;
}
.cat:hover { border-color: var(--accent-dark); }
.cat.active { border-color: var(--accent); background: #134e4a; }

.timer {
  background: var(--panel-2);
  color: var(--accent);
  font-weight: 700;
  min-width: 44px;
  text-align: center;
  padding: 6px 10px;
  border-radius: 50%;
  border: 2px solid var(--accent);
```

```
}
.timer.low { color: var(--bad); border-color: var(--bad); }

.opt {
  display: flex;
  align-items: center;
  gap: 12px;
  text-align: left;
  background: var(--panel-2);
  color: var(--text);
  border: 2px solid #334155;
  border-radius: 12px;
  padding: 13px 14px;
  font-size: .95rem;
  cursor: pointer;
  transition: .2s;
}
.opt:hover:not(:disabled) { border-color: var(--accent); transform: translateX(3px); }

.opt.right { border-color: var(--good); background: rgba(34, 197, 94, .15); }
.opt.wrong { border-color: var(--bad); background: rgba(239, 68, 68, .15); }
.opt.removed { opacity: .25; text-decoration: line-through; }

.ring {
  width: 150px; height: 150px; margin: 18px auto;
  border-radius: 50%;
  display: flex; align-items: center; justify-content: center;
  background: conic-gradient(var(--accent) var(--pct, 0%), var(--panel-2) 0);
  position: relative;
}
.ring::before { content: ""; position: absolute; inset: 12px; background: var(--panel);
border-radius: 50%; }
.ring span { position: relative; font-size: 1.8rem; font-weight: 700; }

@media (max-width: 480px) {
  .card { padding: 20px 16px; }
  .categories { grid-template-columns: 1fr; }
  h1 { font-size: 1.3rem; }
}
```

JavaScript

JavaScript is the brain of the application. The bank object stores the questions of the three categories. shuffle() reorders an array using the Fisher–Yates method. startQuiz() prepares a shuffled copy of the chosen category and re-calculates the index of the correct option after shuffling the choices. loadQuestion() builds the four option buttons with createElement, and startTimer() uses setInterval to count down from 15. answer() locks the question, colours the options, records the result in a history array and updates the score. The 50-50 handler hides two wrong options. showResult() calculates the percentage, saves the best score in localStorage and prints the review list. Only the important parts are shown.

```
// ----- Question bank (one category shown) -----
const bank = {
  ai: [
    { q: "What does AI stand for?", o: ["Automated Input", "Artificial Intelligence",
"Applied Internet", "Advanced Interface"], a: 1 },
    { q: "Which of these is a type of machine learning?", o: ["Supervised learning",
"Compiled learning", "Static learning", "Linked learning"], a: 0 },
    { q: "What is a dataset used for in machine learning?", o: ["Styling a web page",
"Training and testing a model", "Printing documents", "Sending emails"], a: 1 },
    { q: "Which language is most popular for AI programming?", o: ["HTML", "CSS", "Python",
"SQL"], a: 2 },
```

```
    { q: "ChatGPT and Claude are examples of what?", o: ["Search engines", "Operating
systems", "Large language models", "Databases"], a: 2 },
    { q: "What is a neural network inspired by?", o: ["The human brain", "A computer fan",
"Electric wires", "The internet"], a: 0 }
  ]
  // web: [...], python: [...]  (same format)
};

const TIME_PER_Q = 15;

function shuffle(arr) {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

el("startBtn").addEventListener("click", () => {
  const name = el("username").value.trim();
  if (name.length < 2) {
    el("msg").textContent = "Please enter a name with at least 2 characters.";
    return;
  }
  el("msg").textContent = "";
  userName = name;
  startQuiz();
});

// ---------- Quiz logic ----------
function startQuiz() {
  // shuffle questions and also shuffle options of each question
  questions = shuffle(bank[category]).map(item => {
    const correctText = item.o[item.a];
    const opts = shuffle(item.o);
    return { q: item.q, o: opts, a: opts.indexOf(correctText) };
  });
  index = 0;
  score = 0;
  history = [];
  showScreen("quizScreen");
  loadQuestion();
}

function startTimer() {
  clearInterval(timer);
  timeLeft = TIME_PER_Q;
  paintTimer();
  timer = setInterval(() => {
    timeLeft--;
    paintTimer();
    if (timeLeft <= 0) answer(-1);
  }, 1000);
}

function answer(choice) {
  if (locked) return;
  locked = true;
  clearInterval(timer);

  const item = questions[index];
  const opts = document.querySelectorAll(".opt");
```

```
  opts.forEach((b, i) => {
    b.disabled = true;
    if (i === item.a) b.classList.add("right");
    if (i === choice && choice !== item.a) b.classList.add("wrong");
  });

  let status;
  if (choice === -1) status = "timeout";
  else if (choice === item.a) { status = "correct"; score++; }
  else status = "wrong";

  history.push({
    q: item.q,
    chosen: choice === -1 ? "No answer" : item.o[choice],
    correct: item.o[item.a],
    status
  });

  el("scoreLive").textContent = "Score: " + score;
  el("fiftyBtn").disabled = true;
  el("nextBtn").textContent = index === questions.length - 1 ? "Finish" : "Next";
  el("nextBtn").classList.remove("hidden");
}

// 50-50 lifeline: hide two wrong options (one use per quiz)
el("fiftyBtn").addEventListener("click", () => {
  if (locked || fiftyUsed) return;
  fiftyUsed = true;
  el("fiftyBtn").disabled = true;
  const item = questions[index];
  const wrongIdx = [0, 1, 2, 3].filter(i => i !== item.a);
  shuffle(wrongIdx).slice(0, 2).forEach(i => {
    const b = document.querySelectorAll(".opt")[i];
    b.classList.add("removed");
    b.disabled = true;
  });
});

function showResult() {
  el("fill").style.width = "100%";
  showScreen("resultScreen");

  const total = questions.length;
  const pct = Math.round((score / total) * 100);
  const correct = history.filter(h => h.status === "correct").length;
  const wrong = history.filter(h => h.status === "wrong").length;
  const skipped = history.filter(h => h.status === "timeout").length;

  // save best score for the category
  const best = getBest();
  let newBest = false;
  if (best[category] === undefined || score > best[category]) {
    best[category] = score;
    newBest = true;
    try { localStorage.setItem("aiQuizBest", JSON.stringify(best)); } catch (e) {}
  }
  showBest();  // ... title, stats and answer review are filled in here
```

4.3 Project Execution

How to run the application

- Download or clone the project from GitHub and keep index.html, style.css and script.js in the same folder.
- Double-click index.html to open it in Chrome, Edge or Firefox (or right-click and choose Open with Live Server in VS Code).
- No installation, internet connection, server or database is required.

Working process from input to output

The user types a name, picks a category and presses Start Quiz. The first shuffled question appears with four options and a 15-second timer. Clicking an option stops the timer, colours the correct answer green (and a wrong choice red) and shows the Next button. If the timer reaches zero, the correct answer is shown automatically and the question is counted as timed-out. The user may press 50-50 once to remove two wrong options. After the sixth question the Result screen displays the score ring, the percentage, the answer counts and the review list, and updates the best score. The user can then play again or change the category.

5. Output Screenshots

The following screenshots were taken from the working application.

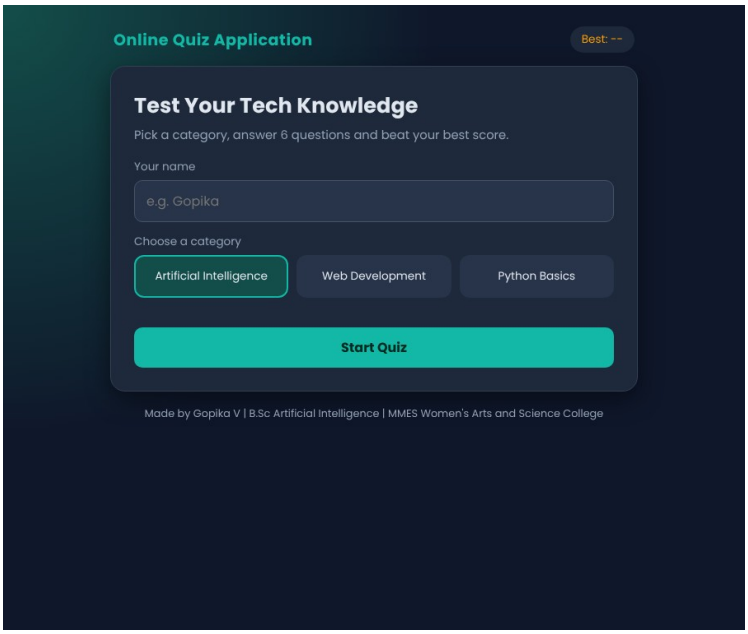


Figure 2: Welcome screen with name box and category buttons

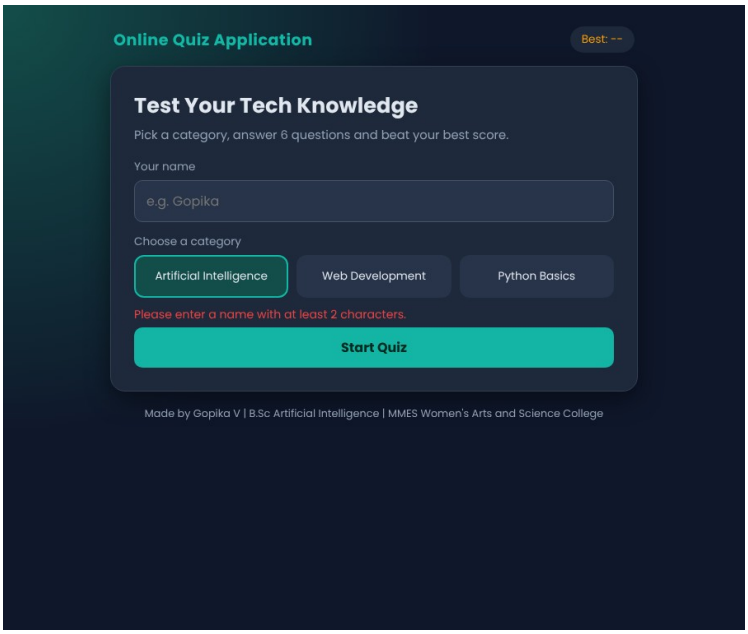


Figure 3: Name validation – error message when the name is empty

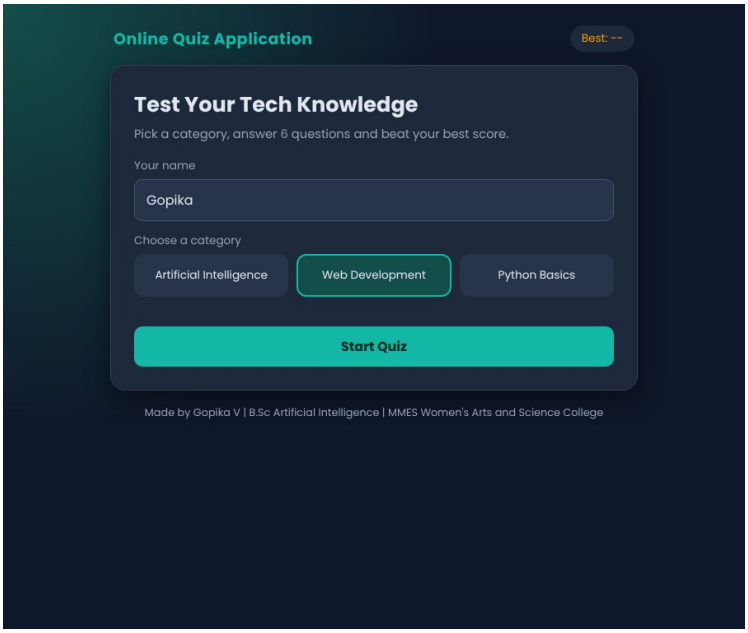


Figure 4: Web Development category selected

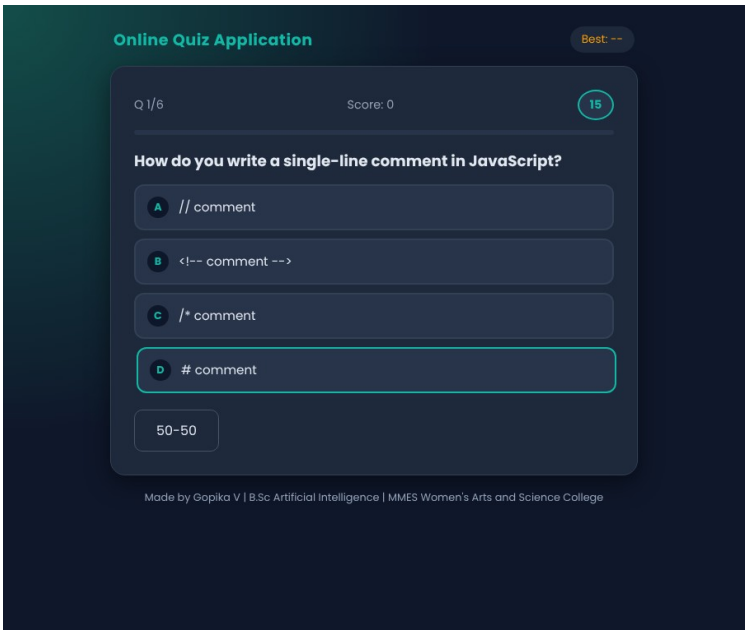


Figure 5: Question screen with counter, score, timer, progress bar and options

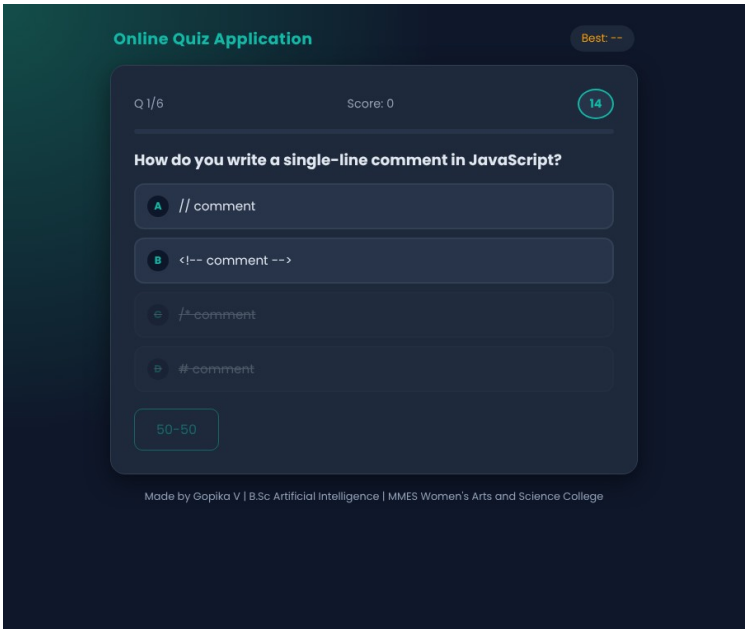


Figure 6: 50-50 lifeline used – two wrong options removed

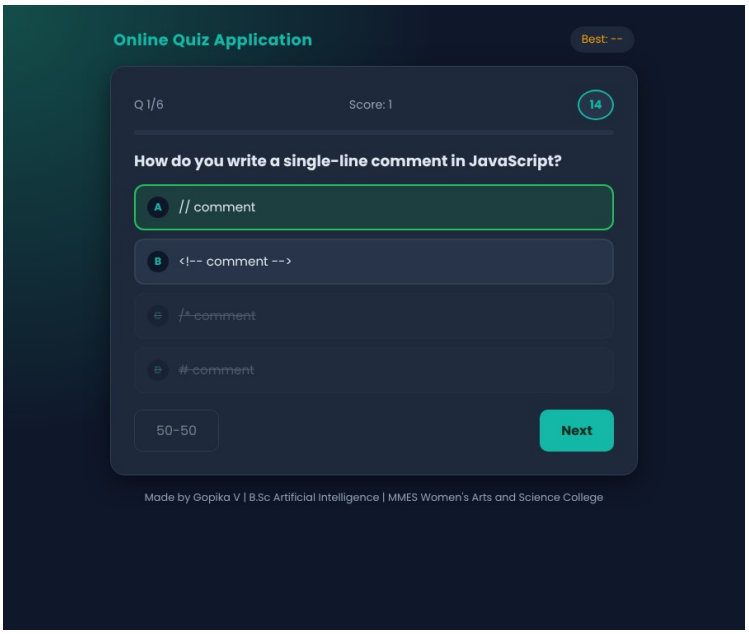


Figure 7: Correct answer selected – option turns green and Next appears

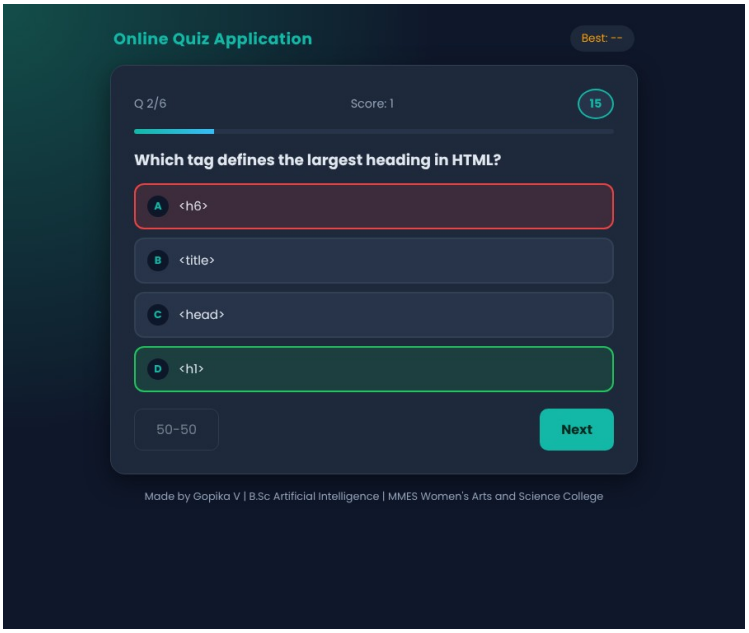


Figure 8: Wrong answer selected – chosen option red, correct option green

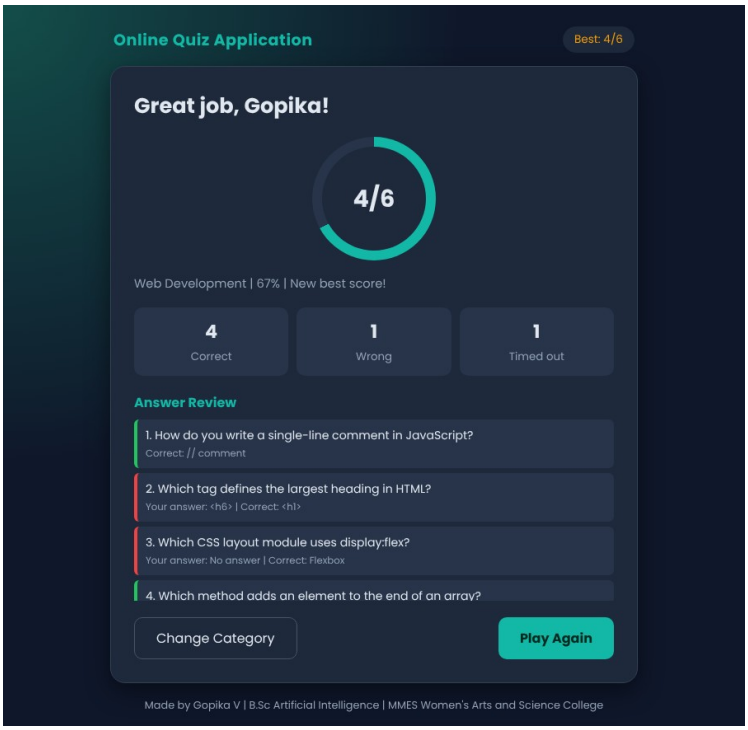


Figure 9: Result screen with score ring, statistics and answer review

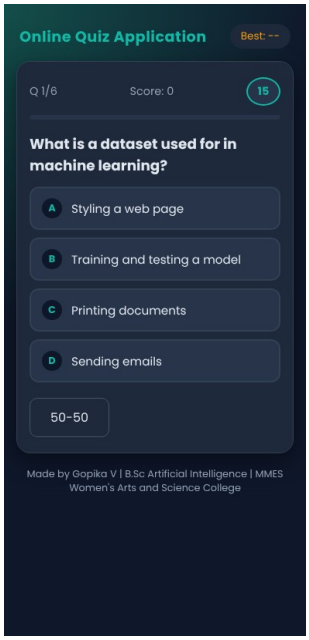


Figure 10: Mobile view (390 px width)

6. Technical Explanation

- **Single-page structure and DOM:** One HTML file holds all screens. getElementById and classList.add/remove switch screens, so the page never reloads.
- **CSS variables and layout:** Theme colours are stored as custom properties; Flexbox aligns the meta bar and buttons, and CSS Grid arranges category buttons and statistics.
- **Data structure:** Each question is an object { q, o, a }. The three categories are arrays inside one bank object and are accessed by a key such as "ai".

- **Shuffling algorithm:** The Fisher–Yates method swaps elements from the end of the array with a random earlier element, giving a fair random order. Because options are shuffled, the position of the correct answer is found again with indexOf().
- **Event handling:** addEventListener is used for category, Start, option, 50-50, Next, Play Again and Change Category buttons.
- **Dynamic DOM creation:** createElement() and forEach() generate the option buttons and the review list items for every question.
- **Timer logic:** setInterval reduces the counter every second; clearInterval stops it when an answer is chosen. A locked flag prevents a question from being scored twice.
- **Conditional logic:** if / else statements validate the name, decide correct, wrong or timed-out, and choose the result message from the percentage.
- **Score calculation:** Percentage = (score ÷ total questions) × 100, rounded with Math.round().
- **Persistent storage:** localStorage keeps the best score of each category as JSON text; try / catch blocks keep the quiz working even if storage is blocked.
- **Security of output:** User text and question text are inserted with textContent, which prevents HTML injection.

7. Conclusion

The Online Quiz Application was developed successfully using HTML, CSS and JavaScript. It lets a student choose a subject, answer shuffled questions against a timer, receive instant feedback, use a lifeline, and finally study a complete review of the attempt. The best score is remembered between visits, and the layout works smoothly on both computers and mobile phones without any installation.

Through this project I learned how to structure a single-page application, design a consistent theme with CSS variables, handle user events, work with arrays and objects, shuffle data, update the DOM dynamically, use timers and store data in the browser. It improved my understanding of how HTML, CSS and JavaScript work together and gave me confidence to build larger web applications.

8. Future Enhancements

- **New features:** more categories and difficulty levels, a larger question bank, negative marking, a choice of quiz length and sound effects.
- **Integration:** a Node.js and MongoDB backend to store questions and results, an admin page to add or edit questions, an API that fetches questions online, and an AI model that suggests revision topics from wrong answers.
- **Enhanced security:** student login, server-side score checking so that answers are not visible in the browser code, and detection of tab switching during a test.
- **Improved user experience:** light / dark mode switch, animations, Tamil and English language options, accessibility support for screen readers and a downloadable certificate for high scorers.